
DCNv2

David Yuan

December 22, 2025

1 EMBEDDING LAYER

The embedding layer serves the starting point, in particular, when dealing with the categorical features. For the i -th feature, it will be projected from high-dimensional sparse space to lower-dimensional space via

$$x_{embed,i} = W_{embed,i} e_i$$

where, $e_i = \{0, 1\}^{\nu_i}$ the one-hot encoding vector, and $W_{embed,i}$ is a learned projection matrix. ν_i is the vocab size.

For example, if we have two categorical features: state = ['CA', 'NY', 'NC'], browser = ['Chrome', 'Safari', 'Firefox']. For state 'CA', the one-hot encoding is $e_1 = [1, 0, 0]$, and assume $W_1 \in R^{2,3}$; for browser 'Safari', the one-hot encoding is $e_2 = [0, 1, 0]$ and assume $W_2 \in R^{3,3}$. Assume we obtained the learned weights for W_1, W_2 , and result in $x_1 = [0.2, 0.5]$, $x_2 = [0.3, 0.1, 0.8]$, then $x_0 = [0.2, 0.5, 0.3, 0.1, 0.8]$. In this example, we assign different embedding sizes, for state the size is 2, and for browser the size is 3.

A popular rule-of-thumb used in production (and also in this paper) is:

$$\text{embedding size for feature } i = 6 \times (\text{vocab size})^{1/4}$$

2 DEEP AND CROSS NETWORK

The cross network key is

$$x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$$

where, $\odot$ is the elementwise (Hadamard) product.

The deep network key is:

$$h_{l+1} = f(W_l h_l + b_l)$$

where, $f(\cdot)$ is an elementwise activation function (ReLU in paper).

Note: It is better to be noted that the cross network and deep network weights and bias are trained separately, which are not shared.

3 STACKED ARCHITECTURE - ILLUSTRATION

3.1 CROSS NETWORK (2 LAYERS)

$$\mathbf{x}_1 = \mathbf{x}_0 \odot (W_0^{\text{cross}} \mathbf{x}_0 + \mathbf{b}_0^{\text{cross}}) + \mathbf{x}_0 \quad (3.1)$$

$$\mathbf{x}_2 = \mathbf{x}_0 \odot (W_1^{\text{cross}} \mathbf{x}_1 + \mathbf{b}_1^{\text{cross}}) + \mathbf{x}_1 \quad (3.2)$$

3.2 DEEP NETWORK (2 LAYERS)

Let the input to the deep network be:

$$\mathbf{h}_0 = \mathbf{x}_2$$

Then apply 2 ReLU layers:

$$\mathbf{h}_1 = \text{ReLU}(W_0^{\text{deep}} \mathbf{h}_0 + \mathbf{b}_0^{\text{deep}}) \quad (3.3)$$

$$\mathbf{h}_2 = \text{ReLU}(W_1^{\text{deep}} \mathbf{h}_1 + \mathbf{b}_1^{\text{deep}}) \quad (3.4)$$

3.3 OUTPUT LAYER (SIGMOID)

$$\hat{y} = \sigma(\mathbf{w}_{\text{logit}}^\top \mathbf{h}_2)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$

3.4 LOSS FUNCTION (BINARY CROSS-ENTROPY)

Assuming the target $y = 1$, the loss is:

$$\mathcal{L} = -\log(\hat{y})$$

4 PARALLEL ARCHITECTURE - ILLUSTRATION

4.1 CROSS NETWORK (2 LAYERS)

$$\mathbf{x}_1 = \mathbf{x}_0 \odot (W_0^{\text{cross}} \mathbf{x}_0 + \mathbf{b}_0^{\text{cross}}) + \mathbf{x}_0 \quad (4.1)$$

$$\mathbf{x}_2 = \mathbf{x}_0 \odot (W_1^{\text{cross}} \mathbf{x}_1 + \mathbf{b}_1^{\text{cross}}) + \mathbf{x}_1 \quad (4.2)$$

4.2 DEEP NETWORK (2 LAYERS)

$$\mathbf{h}_0 = \mathbf{x}_0 \quad (4.3)$$

$$\mathbf{h}_1 = \text{ReLU} \left(W_0^{\text{deep}} \mathbf{h}_0 + \mathbf{b}_0^{\text{deep}} \right) \quad (4.4)$$

$$\mathbf{h}_2 = \text{ReLU} \left(W_1^{\text{deep}} \mathbf{h}_1 + \mathbf{b}_1^{\text{deep}} \right) \quad (4.5)$$

4.3 CONCATENATION AND PREDICTION

$$\mathbf{x}_{\text{final}} = [\mathbf{x}_2; \mathbf{h}_2] \quad (4.6)$$

$$\hat{y} = \sigma \left(\mathbf{w}_{\text{logit}}^\top \mathbf{x}_{\text{final}} \right) \quad (4.7)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

4.4 LOSS FUNCTION (BINARY CROSS-ENTROPY)

If the ground truth label is $y = 1$, then:

$$\mathcal{L} = -\log(\hat{y})$$

5 PARAMETERS TRAINED

5.1 NOTATION

- m : Number of categorical features (fields)
- n_i : Number of unique categories in field i
- d_i : Embedding dimension for field i
- d : Output embedding size (total after concatenation), also input size to cross/deep nets
- L_c : Number of cross layers
- L_d : Number of deep layers
- h : Width of each hidden layer in the deep network

5.2 EMBEDDING LAYER PARAMETERS

For each categorical feature i :

$$\text{Embedding params} = \sum_{i=1}^m n_i \cdot d_i$$

If all embeddings use the same dimension d and each feature has n categories:

$$\text{Embedding params} = m \cdot n \cdot d$$

5.3 CROSS NETWORK PARAMETERS

Each cross layer:

$$\mathbf{x}_{l+1} = \mathbf{x}_0 \odot (W_l \mathbf{x}_l + \mathbf{b}_l) + \mathbf{x}_l$$

with:

$$W_l \in \mathbb{R}^{d \times d}, \quad \mathbf{b}_l \in \mathbb{R}^d$$

So total:

$$\text{Cross params} = L_c \cdot (d^2 + d)$$

5.4 DEEP NETWORK PARAMETERS

Let input to deep network be $\mathbf{x}_0 \in \mathbb{R}^d$, each hidden layer has width h .

- First layer: $W_0 \in \mathbb{R}^{h \times d}, \quad \mathbf{b}_0 \in \mathbb{R}^h$
- Remaining $L_d - 1$ layers: $W_l \in \mathbb{R}^{h \times h}, \quad \mathbf{b}_l \in \mathbb{R}^h$

$$\text{Deep params} = (h \cdot d + h) + (L_d - 1) \cdot (h^2 + h)$$

5.5 LOGIT LAYER PARAMETERS

Input: concatenation of cross and deep outputs $\in \mathbb{R}^{d+h}$

$$\text{Logit params} = (d + h) + 1$$

5. TOTAL PARAMETERS

$$\text{Total} = \left(\sum_{i=1}^m n_i \cdot d_i \right) + L_c \cdot (d^2 + d) + (h \cdot d + h) + (L_d - 1)(h^2 + h) + (d + h + 1)$$

EXAMPLE:

Let:

$$m = 10, \quad n_i = 1000, \quad d_i = 16, \quad d = 128, \quad h = 256, \quad L_c = 2, \quad L_d = 2$$

Then:

$$\text{Embedding} = 10 \cdot 1000 \cdot 16 = 160,000$$

$$\text{Cross} = 2 \cdot (128^2 + 128) = 33,024$$

$$\text{Deep} = (256 \cdot 128 + 256) + (256^2 + 256) = 98,816$$

$$\text{Logit} = 128 + 256 + 1 = 385$$

$$\text{Total} = \boxed{292,225 \text{ parameters}}$$

6 LOW RANK

In the implementation, it is not implemented by the full weights matrix, but instead implementation of the low-rank structure, that is,

$$\mathbf{x}_{l+1} = \mathbf{x}_0 \odot (U_l (V_l^\top \mathbf{x}_l) + \mathbf{b}_l) + \mathbf{x}_l$$

where, $U_l, V_l \in R^{d \times r}$, $r \ll d$, and hyper-parameter r is fine-tuned. For example, if $d = 512$, $r = 64$, then the new number of parameters is $512 \times 64 = 32768$, while the old number of parameters is $512^2 = 262144$, a 87% reduction.